

LangChain

Conversation Memory

[BufferMemory](#) · [SummaryMemory](#) · [Token Comparison](#) · [When to Use Each](#)

[Fundamentals](#)[BufferMemory](#)[SummaryMemory](#)[Token Cost](#)[Decision Guide](#)

1. Why Do AI Chatbots Need Memory?

By default, every call to a language model is **completely stateless**. The model has no memory of what was said before. Each API call is like meeting someone for the first time — it has no idea what you discussed five seconds ago.

To build a chatbot that remembers the conversation, you must **manually include the previous messages in every new API call**. LangChain's memory classes automate this — they store the conversation history and inject it into each new prompt so the model always has context.

The core problem: stateless vs. stateful

	Without Memory	With Memory
Turn 1	User: My name is Alice	User: My name is Alice
Turn 2	User: What is my name? AI: I don't know your name.	User: What is my name? AI: Your name is Alice.
Why different?	Model saw only Turn 2 prompt	Model saw Turn 1 + Turn 2 in context

How LangChain memory works internally

Every LangChain memory class follows the same two-step pattern on every turn:

- ① **Step 1 — Load:** Before sending the prompt to the model, the memory class reads stored history and injects it into the prompt automatically.
- ② **Step 2 — Save:** After the model replies, the memory class saves the new human message and AI response to its internal store.

The two memory types covered in this guide

Memory Type	Strategy	Best For
ConversationBufferMemory	Stores every message verbatim	Short conversations (≤ 10 turns)
ConversationSummaryMemory	Summarises older history with LLM	Long conversations (15+ turns)

2. ConversationBufferMemory — Fundamentals

ConversationBufferMemory is the simplest memory type. It stores every single message in the conversation — word for word — and sends the entire history to the model on every new turn. Think of it as a perfect transcript that grows with every exchange.

Visual: what gets sent on each turn

Turn	What is sent to the model (context window)
1	[Human: Hello, my name is Alice]
2	[Human: Hello, my name is Alice] [AI: Hi Alice! Nice to meet you.] [Human: I love Python.]
3	... + [AI: Great choice!] + [Human: What language did I say I love?]
10	All 9 previous exchanges + current question — context keeps growing

Key properties

- **Perfect recall:** Remembers every word exactly as spoken — nothing is lost or paraphrased.
- **Growing token cost:** Each turn adds more tokens. Turn 15 sends ~15x more tokens than Turn 1.
- **Context window limit:** If the conversation grows too long it eventually hits the model's token limit and fails.
- **Zero LLM calls:** No extra API calls to maintain memory — it's just string concatenation.
- **memory_key param:** Controls the variable name injected into the prompt (default: 'history').

- BufferMemory is the right choice when your conversations are short (under 10–15 turns) and you need the model to recall precise details — exact numbers, names, or phrasing from earlier in the chat.

3. BufferMemory — 5-Turn Conversation (Code)

EXAMPLE 1 — BufferMemory: 5-turn conversation

[illegible]

```

response1 = chain.invoke({"input": "Hi! My name is Alice and I am a data scientist."})
print(f"AI: {response1['response']}")

print("\n" + "=" * 60)
print("TURN 2")
response2 = chain.invoke({"input": "I love working with Python and pandas."})
print(f"AI: {response2['response']}")

print("\n" + "=" * 60)
print("TURN 3")
response3 = chain.invoke({"input": "My favourite project was building a churn
prediction model."})
print(f"AI: {response3['response']}")

print("\n" + "=" * 60)
print("TURN 4 – testing memory: asking about Turn 1 info")
response4 = chain.invoke({"input": "What is my name and what do I do?"})
print(f"AI: {response4['response']}")
# Expected: AI should recall 'Alice' and 'data scientist' from Turn 1

print("\n" + "=" * 60)
print("TURN 5 – testing memory: asking about Turn 2 and 3 info")
response5 = chain.invoke({"input": "What programming language do I use and what was my
favourite project?"})
print(f"AI: {response5['response']}")
# Expected: AI should recall 'Python/pandas' and 'churn prediction model'

```

OUTPUT

```

=====
TURN 1
AI: Hi Alice! Great to meet you. Data science is such an exciting field!

=====
TURN 2
AI: Python and pandas are a fantastic combination for data work!

=====
TURN 3
AI: That sounds like a great project! Churn prediction is really valuable.

=====
TURN 4 – testing memory
AI: Your name is Alice and you are a data scientist!
[✓ Correctly recalled from Turn 1]

=====
TURN 5 – testing memory
AI: You work with Python and pandas, and your favourite project was
building a churn prediction model!

```

[✓ Correctly recalled from Turn 2 and Turn 3]

Inspecting what the memory holds

EXAMPLE 2 — Inspecting the buffer after 5 turns

```
# ■■■ Inspect the stored memory after 5 turns ■■■■■■■■■■■■■■■■■■■■■■
```

```
# memory.chat_memory.messages returns the full list of Message objects
```

```
print("\n■■■ Memory contents after 5 turns ■■■")
```

```
for i, msg in enumerate(memory.chat_memory.messages):
```

```
# Each message has a 'type' ('human' or 'ai') and 'content'
```

```
role = "Human" if msg.type == "human" else "AI"
```

```
# Truncate long messages for display
```

```
content_preview = msg.content[:80] + "..." if len(msg.content) > 80 else msg.content
```

```
print(f"[{i+1}] {role}: {content_preview}")
```

```
# Count total messages stored
```

```
total_messages = len(memory.chat_memory.messages)
```

```
print(f"\nTotal messages in buffer: {total_messages}")
```

```
# Expected: 10 messages (5 human + 5 AI responses)
```

```
# Load memory as a dict – this is what gets injected into the prompt
```

```
loaded = memory.load_memory_variables({})
```

```
print(f"Memory key: {list(loaded.keys())}") # ['history']
```

```
print(f"Type of history: {type(loaded['history'])}") # list of Message objects
```

4. ConversationSummaryMemory — Fundamentals

ConversationSummaryMemory solves the biggest problem with BufferMemory: the growing token cost. Instead of storing every message verbatim, it uses the LLM to **periodically summarise older parts of the conversation** into a compact paragraph. Only the summary + recent messages are kept in context.

Think of it like meeting notes — instead of reading the full 3-hour transcript, you read a 1-paragraph summary of what was discussed, then the last few minutes verbatim.

Visual: what gets sent on each turn

Turn	What is sent to the model
1–4	Full verbatim history (short enough to keep as-is)
5	[Summary: 'Alice is a data scientist who loves Python...'] + [Human: Turn 5 message]
10	[Summary: updated, covering Turns 1-9 in a paragraph] + [Human: Turn 10 message]
20	[Summary: ~300 tokens covering all 19 prior turns] + [Human: Turn 20 message] ← token count stays roughly constant!

Key properties

- **Controlled token growth:** Context size stays roughly constant regardless of conversation length.
- **Requires extra LLM calls:** Each summarisation step calls the LLM — adds small latency and cost.
- **Lossy compression:** Fine details from older turns may be lost or paraphrased. Not pixel-perfect recall.
- **Graceful degradation:** As the conversation grows, older details are preserved as concepts, not exact quotes.
- **max_token_limit param:** Controls when summarisation triggers. Default varies; tune to your use case.

- SummaryMemory is the right choice for long conversations (15+ turns), customer support bots, tutoring assistants, or anywhere the chat may run indefinitely. It keeps the token cost predictable while preserving the gist of everything said.

5. SummaryMemory — 15-Turn Conversation (Code)

EXAMPLE 3 — SummaryMemory: 15-turn road trip conversation

[illegible]


```

"I love history and quirky roadside attractions.",
"I prefer staying at motels rather than hotels.",
"I'm vegetarian, so I'll need food options along the way.",
"What are the must-see stops in Missouri?",
"What about Oklahoma – any hidden gems there?",
"I'm a bit nervous about driving through the Texas panhandle.",
"What's the best way to handle car maintenance for a long trip?",
"How much should I budget per day for food and fuel?",
"What apps would you recommend for a solo road tripper?",
"Summarise everything we've discussed so far about my trip." # Turn 15: test recall
]

print("Running 15-turn conversation with SummaryMemory...\n")
responses = []
for i, user_input in enumerate(turns, start=1):
    response = chain.invoke({"input": user_input})
    responses.append(response["response"])
    print(f"Turn {i:2d} | User: {user_input[:60]}...")
    print(f" | AI : {response['response'][:80]}...")
    print()

```

EXAMPLE 4 — Inspecting memory state after 15 turns

```

# ■■ After all 15 turns: inspect what the memory looks like ■■■■■■■■

print("\n" + "=" * 60)
print("MEMORY INSPECTION AFTER 15 TURNS")
print("=" * 60)

# Load the current memory state
loaded_memory = memory.load_memory_variables({})

# SummaryMemory stores a single summary string (not individual messages)
# The 'history' key contains a SystemMessage with the running summary
print("\nMemory type:", type(memory))
print("\nCurrent summary stored in memory:")
print("-" * 40)

# The summary is in memory.moving_summary_buffer
print(memory.moving_summary_buffer)

print("-" * 40)

# Note: the summary should mention New York, LA, Route 66,
# $3000 budget, 14 days, Honda Civic, history lover,
# vegetarian, motels – all compressed into a paragraph!

# Count messages in the summary buffer
messages = memory.chat_memory.messages

```

```
print(f"\nMessages in raw chat_memory: {len(messages)}")
print("(Only very recent messages are kept – older ones are summarised)")

# Show the final Turn 15 response
print("\nTurn 15 AI response (should recall all key trip details):")
print(responses[14])
```

OUTPUT

```
MEMORY INSPECTION AFTER 15 TURNS
=====
Current summary stored in memory:
-----
The human is planning a solo road trip from New York to Los Angeles on
Route 66 over 14 days with a $3,000 budget, driving a Honda Civic.
They enjoy history and quirky roadside attractions, prefer staying at
motels, and are vegetarian. They discussed must-see stops in Missouri
and Oklahoma, tips for driving the Texas panhandle, car maintenance
advice, a daily budget of ~$150-200 for food and fuel, and recommended
apps including GasBuddy, iOverlander, and HappyCow for vegetarian food.

Messages in raw chat_memory: 2
(Only recent messages kept – older ones are summarised)

Turn 15 AI response:
"You're planning a solo 14-day road trip from NYC to LA on Route 66
with a $3,000 budget in your Honda Civic. You love history and quirky
stops, prefer motels, and are vegetarian..."
[✓ Correctly recalled all key details from all 15 turns]
```

The full conversation is compressed into ~200 tokens instead of ~2000

This is the most important practical difference between the two memory types. Here is how to measure token usage and what you can expect to see.

[illegible]

[illegible]

```
f"{buffer_grand_total - summary_grand_total:>+12,}")
savings_pct = (buffer_grand_total - summary_grand_total) / buffer_grand_total * 100
print(f"\n SummaryMemory saved {savings_pct:.1f}% of total tokens over 15 turns")
```

OUTPUT

```
COMPARISON AFTER 15 TURNS
=====
Turn BufferMemory SummaryMemory Difference
-----
1 312 312 0
2 384 384 0
3 468 412 +56
4 554 445 +109
5 643 478 +165
6 735 509 +226
7 832 534 +298
8 934 561 +373
9 1,042 589 +453
10 1,157 618 +539
11 1,279 648 +631
12 1,408 679 +729
13 1,545 711 +834
14 1,690 744 +946
15 1,843 779 +1,064
-----
TOTAL 14,830 8,403 +6,427

SummaryMemory saved 43.3% of total tokens over 15 turns

(Note: exact numbers depend on model and conversation length)
```

BufferMemory token cost grows linearly; SummaryMemory grows much more slowly

Why the gap grows with every turn

Each BufferMemory turn sends the *entire history* again — so Turn 15 sends Turn 1 through 14 in full. SummaryMemory replaces that history with a compact paragraph, so the context size grows much more slowly. The savings compound: the longer the conversation, the bigger the gap.

7. When to Choose BufferMemory vs SummaryMemory

The right memory type depends on your use case. Here is a practical decision guide based on conversation length, recall needs, and cost.

Factor	Choose BufferMemory	Choose SummaryMemory
Conversation length	Short — up to 10–15 turns	Long — 15+ turns, or unbounded
Recall precision	Need exact word-for-word recall	Conceptual recall is sufficient
Token budget	Flexible / cost not critical	Must control token cost
Latency requirements	Need fastest possible response	Can tolerate slightly higher latency
Extra LLM calls OK?	Yes — no extra calls	Yes — summary generation adds ~1 call
Use case examples	Quick Q&A bots, coding assistants, short customer queries	Customer support, tutoring bots, long research sessions
Risk	Context window overflow after ~20 turns	Older details may be paraphrased/lost
Cost per 15-turn session	Higher (~1.8x SummaryMemory)	Lower (~43% saving typical)

Simple decision flowchart

Question	Answer	Decision
Will conversation exceed 15 turns?	No	→ Use BufferMemory
Will conversation exceed 15 turns?	Yes / Maybe	→ Continue below
Do you need exact word-for-word recall?	Yes	→ Use BufferMemory (+ handle context overflow)
Do you need exact word-for-word recall?	No	→ Use SummaryMemory
Is token cost a concern?	Yes	→ Use SummaryMemory
Is token cost a concern?	No	→ Either works; Buffer is simpler

- **Hybrid approach:** For production systems, consider **ConversationSummaryBufferMemory** — it keeps recent messages verbatim (exact recall for recent turns) and summarises older messages (bounded token cost for long conversations). Best of both worlds.

8. ConversationSummaryBufferMemory — Best of Both Worlds

You now know the two extremes:

- **BufferMemory** — keeps everything verbatim. Perfect recall, but token cost grows without limit.
- **SummaryMemory** — compresses everything into a summary. Bounded cost, but loses fine detail from older turns.

ConversationSummaryBufferMemory sits exactly in the middle. It keeps recent messages *word for word* (so you have precise recall of what was just said), and once the stored messages exceed a token limit you set, it automatically summarises the oldest ones into a compact paragraph. The summary is then kept alongside the recent verbatim messages.

The notebook analogy

Imagine a student taking notes in a lecture. For the **last 5 minutes** they write down every word verbatim — because those details are fresh and still relevant. For **everything before that**, they condense their earlier pages into a bullet-point summary at the top. The notebook never gets full, but they still have exact notes for what just happened.

How it works step by step

Step	What happens
1 — New message arrives	The new human message is added to the buffer (verbatim store).
2 — Check token count	Count the tokens of all messages currently in the buffer.
3a — Under the limit	Token count ≤ max_token_limit: send everything verbatim. No summarisation yet.
3b — Over the limit	Token count > max_token_limit: take the OLDEST messages, summarise them with the LLM, store in summary.
4 — What gets sent to model	[Summary of old turns] + [Recent turns verbatim] + [New message]

Visual: what gets sent to the model on each turn

Turn	max_token_limit = 300	What is sent
1–4	Buffer: 240 tokens (under limit)	All 4 turns verbatim — no summary yet
5	Buffer: 340 tokens (OVER limit)	[Summary of Turns 1–2] + [Turns 3–4 verbatim] + [Turn 5]
10	Buffer grows, triggers again	[Summary of Turns 1–7] + [Turns 8–9 verbatim] + [Turn 10]
20	Summary keeps updating	[Summary of Turns 1–17] + [Turns 18–19 verbatim] + [Turn 20]

- **max_token_limit** is the only new parameter you need to learn. It controls the threshold: once the verbatim buffer exceeds this many tokens, the oldest messages get summarised. A good starting value is **600–1000 tokens** — enough to keep 4–6 recent turns verbatim while summarising everything older.

Example: SummaryBufferMemory in action

EXAMPLE 6 — SummaryBufferMemory: 10-turn conversation

```
#
# EXAMPLE 6: ConversationSummaryBufferMemory – hybrid memory
# Keeps recent messages verbatim + summarises older ones.
#
from langchain.memory import ConversationSummaryBufferMemory
from langchain.chains import ConversationChain
from langchain_openai import ChatOpenAI

# ■■ Step 1: Create the model
llm = ChatOpenAI(
    model="gpt-4o-mini",
    temperature=0.7,
    api_key="YOUR_OPENAI_API_KEY"
)

# ■■ Step 2: Create SummaryBufferMemory
# max_token_limit: once the verbatim buffer exceeds this many tokens,
# the oldest messages are summarised and replaced with a summary.
# 300 is a low value to trigger summarisation quickly for this demo.
memory = ConversationSummaryBufferMemory(
    llm=llm, # used to generate summaries
    max_token_limit=300, # trigger summarisation above 300 tokens
    memory_key="history", # variable injected into the prompt
    return_messages=True # needed for chat models
)

# ■■ Step 3: Build the chain
chain = ConversationChain(
    llm=llm,
    memory=memory,
    verbose=False
)

#
# Run 10 turns – track when summarisation kicks in
#
turns = [
    "Hi! I'm Alice and I'm a Python developer.", # Turn 1
    "I'm working on a machine learning project.", # Turn 2
    "I'm using scikit-learn and pandas.", # Turn 3
    "The project is about predicting house prices.", # Turn 4
    "I have 10,000 rows of training data.", # Turn 5
```



```

=====
[A] Summary of older turns:
-----

Alice is a Python developer working on a house price prediction
project using scikit-learn, pandas, and a Random Forest model.
She has 10,000 training rows and is achieving an R2 score of 0.78,
and is looking for ways to improve performance.

[B] Verbatim messages still in buffer:
-----

[Human]: Which model would you recommend trying next?...
[AI]: I'd suggest trying Gradient Boosting (XGBoost/LightGBM)...
[Human]: What was my name and what project am I working on?...
[AI]: Your name is Alice, and you are working on a house price
prediction project using scikit-learn and Random Forest...
[✓ Recalled from summary – not verbatim buffer!]

[C] Combined context sent to model:
-----

Type: system | Summary: Alice is a Python developer...
Type: human | Content: Which model would you recommend...
Type: ai | Content: I'd suggest trying Gradient Boosting...

```

Older turns are in the summary; recent turns are verbatim — both accessible to the model

Three-way comparison: Buffer vs Summary vs SummaryBuffer

	BufferMemory	SummaryMemory	SummaryBufferMemory
What is stored	All messages verbatim	One running summary	Summary + recent verbatim
Exact recent recall	✓ Yes	✗ May be paraphrased	✓ Yes (recent turns)
Exact old recall	✓ Yes	✗ Compressed	✗ Compressed into summary
Token cost (long)	Grows linearly	Nearly flat	Grows slowly then flattens
Extra LLM calls	None	Yes — each summary step	Yes — when limit exceeded
Best for	Short chats, exact quotes	Very long / infinite chats	Production chatbots (best balance)
Key parameter	—	—	max_token_limit

- **SummaryBufferMemory is the recommended default for production chatbots.** It gives you exact recall of the last few turns (so the model never forgets what the user just said) while keeping token cost bounded for long conversations. Start with **max_token_limit=600** and tune based on your use case.

Summary

Concept	Key Point
Why memory is needed	LLMs are stateless — history must be manually injected each turn.
ConversationBufferMemory	Stores all messages verbatim. Perfect recall, but grows linearly.
ConversationSummaryMemory	Summarises older history with LLM. Bounded token cost, slight loss of detail.
How chains work	ConversationChain wraps prompt + memory + llm. Auto loads/saves history.
invoke() input	chain.invoke({'input': 'user message'}) — always the same key.
BufferMemory token cost	Grows with every turn — ~linear increase in context size.
SummaryMemory token cost	Grows slowly — older turns compressed into a summary paragraph.
At 15 turns	SummaryMemory typically saves 30–50% total tokens vs BufferMemory.
Choose BufferMemory when	Short conversations, need exact recall, cost not critical.
Choose SummaryMemory when	Long/unbounded conversations, cost matters, conceptual recall is OK.
SummaryBufferMemory	Keeps recent messages verbatim + summarises older ones when limit exceeded.
max_token_limit param	Controls when SummaryBufferMemory triggers summarisation. Start at 600.
What gets sent (hybrid)	[Summary of old turns] + [Recent turns verbatim] + [New message]
Recommended default	Use SummaryBufferMemory for most production chatbots — best balance.

- Start with **Example 1** (5-turn BufferMemory), then **Example 3** (15-turn SummaryMemory), then **Example 6** (hybrid SummaryBufferMemory). Run **Example 5** to see the token gap. Use Section 7 to pick the right type.